

Лабораторная работа №7

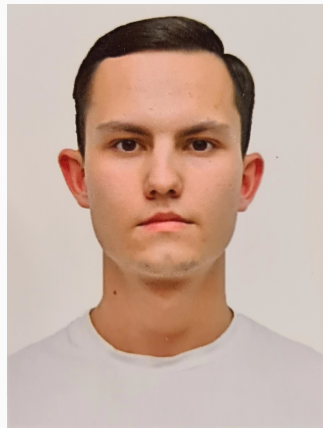
Введение в работу с данными

Дурдалыев Максат

2025-12-06

Российский университет дружбы народов имени Патриса Лумумбы, Москва, Россия

- Дурдалыев Максат
- Студент НКНбд-01-22
- Российский университет дружбы народов имени Патриса Лумумбы
- 1132205337@pfur.ru
- <https://github.com/mdurdalyyev>



Цель работы

Основной целью работы является специализированных пакетов Julia для обработки данных.

Задание

1. Используя Jupyter Lab, повторите примеры.
2. Выполните задания для самостоятельной работы.

3 Считывание данных

В Julia для работы со структурами данных используют пакеты CSV, DataFrames, RDatasets, FileIO:

```
using Pkg
```

```
[1] ✓ 3.0s
```

Julia



```
for p in ["CSV", "DataFrames", "RDatasets", "FileIO"]  
    Pkg.add(p)  
end
```

```
[ ]
```

Julia

4 Считывание данных

```
# Считывание данных и их запись в структуру:  
P = CSV.File("programminglanguages.csv") |> DataFrame
```

Julia

```
# Функция определения по названию языка программирования года его создания:  
function language_created_year(P, language::String)  
    loc = findfirst(P[:,2].==language)  
    return P[loc,1]  
end
```

✓ 0.1s

Julia

language_created_year (generic function with 1 method)

```
# Пример вызова функции и определение даты создания языка Python:  
language_created_year(P, "Python")
```

✓ 0.2s

Julia

5 Считывание данных

```
language_created_year(P,"julia")
1.1s
MethodError: no method matching getindex(::DataFrame, ::Nothing, ::Int64)
The function `getindex` exists, but no method is defined for this combination of argument types.

Closest candidates are:
  getindex(::DataFrame, !Matched::typeof(!), ::Union{Signed, Unsigned})
    @ DataFrames C:\Users\durda\.julia\packages\DataFrames\b4w9K\src\dataframe\dataframe.jl:548
  getindex(::DataFrame, !Matched::Colon, ::Union{AbstractString, Signed, Symbol, Unsigned})
    @ DataFrames C:\Users\durda\.julia\packages\DataFrames\b4w9K\src\dataframe\dataframe.jl:542
  getindex(::DataFrame, !Matched::Integer, ::Union{Signed, Unsigned})
    @ DataFrames C:\Users\durda\.julia\packages\DataFrames\b4w9K\src\dataframe\dataframe.jl:586
  ...

Stacktrace:
 [1] language_created_year(P::DataFrame, language::String)
    @ Main d:\V\ve6a\4 kypc\Computer practice\la6a 7\jl_notebook_cell_df34fa98e69747e1a8f8a738347b8e2f_X24s2mlsZQ=.jl:4
 [2] top-level scope
    @ d:\V\ve6a\4 kypc\Computer practice\la6a 7\jl_notebook_cell_df34fa98e69747e1a8f8a738347b8e2f_X21s2mlsZQ=.jl:1
```

Рисунок 4: Поиск «julia» со строчной буквы

6 Считывание данных

```
# Функция определения по названию языка программирования  
# года его создания (без учёта регистра):  
function language_created_year_v2(P, language::String)  
    loc = findfirst(lowercase.(P[:,2]).==lowercase.(language))  
    return P[loc,1]  
end
```

✓ 0.0s

Julia

language_created_year_v2 (generic function with 1 method)

```
# Пример вызова функции и определение даты создания языка julia:  
language_created_year_v2(P, "julia")
```

✓ 0.1s

Julia

7 Считывание данных

```
# Построчное считывание данных с указанием разделителя:
```

```
Tx = readln("programminglanguages.csv", ',')
```

```
✓ 1.6s
```

Julia

```
74x2 Matrix{Any}:
```

```
  "year"  "language"
```

```
1951      "Regional Assembly Language"
```

```
1952      "Autocode"
```

```
1954      "IPL"
```

```
1955      "FLOW-MATIC"
```

```
1957      "FORTRAN"
```

```
1957      "COMTRAN"
```

```
1958      "LISP"
```

```
1958      "ALGOL 58"
```

```
1959      "FACT"
```

```
⋮
```

```
2007      "Clojure"
```

```
2009      "Go"
```

```
2010      "Rust"
```

```
2011      "Dart"
```

```
2011      "Kotlin"
```

```
2011      "Red"
```

```
2011      "Elixir"
```

```
2012      "Julia"
```

```
2014      "Swift"
```

8 Запись данных в файл

```
# Запись данных в CSV-файл:
```

```
CSV.write("programming_languages_data2.csv", P)
```

```
✓ 0.0s
```

julia

```
"programming_languages_data2.csv"
```

9 Запись данных в файл

```
# Построчное считывание данных с указанием разделителя:  
P_new_delim = readlm("programming_languages_data2.txt", '-')
```

✓ 0.0s

Julia

74x2 Matrix{Any}:

"year"	"language"
1951	"Regional Assembly Language"
1952	"Autocode"
1954	"IPL"
1955	"FLOW-MATIC"
1957	"FORTRAN"
1957	"COMTRAN"
1958	"LISP"
1958	"ALGOL 58"
1959	"FACT"
:	
2007	"Clojure"
2009	"Go"
2010	"Rust"
2011	"Dart"
2011	"Kotlin"
2011	"Red"
2011	"Elixir"
2012	"Julia"
2014	"Swift"

Инициализация словаря:

```
dict = Dict{Integer, Vector{String}}()
```

✓ 0.5s

julia

```
Dict{Integer, Vector{String}}()
```

11 Словари

```
# Заполнение словаря данными:  
for i = 1:size(P,1)  
    year,lang = P[i,:]   
    if year in keys(dict)  
        dict[year] = push!(dict[year],lang)  
    else  
        dict[year] = [lang]  
    end  
end
```

✓ 0.1s

Julia

13/49

12 DataFrames

```
# Сноваим переименование со сгруппированных DataFrames:
df = DataFrame(year = P[:,1], language = P[:,2])
```

✓ 0.0s

73x2 DataFrame 48 rows omitted

Row	year	language
	Int64	String31
1	1951	Regional Assembly Language
2	1952	Autocode
3	1954	IPL
4	1955	FLOW-MATIC
5	1957	FORTRAN
6	1957	COMTRAN
7	1958	LISP
8	1958	ALGOL 58
9	1959	FACT
10	1959	COBOL
11	1959	RPG
12	1962	APL
13	1962	Simula
⋮	⋮	⋮
62	2003	Scala
63	2005	F#
64	2006	PowerShell
65	2007	Clojure
66	2009	Go
67	2010	Rust
68	2011	Dart
69	2011	Kotlin

Рисунок 14: Пример создания структуры DataFrame

13 DataFrames

```
# Вывод всех значения столбца year:
df[1,:year]
✓ 0.2s
```

73-element Vector{Int64}:
1951
1952
1954
1955
1957
1957
1958
1958
1959
1959
:
2007
2009
2010
2011
2011
2011
2011
2012
2014

```
# Получение статистических сведений о фрейме:
describe(df)
✓ 0.8s
```

2x7 DataFrame

Row	variable	mean	min	median	max	nmissing	eltype
	Symbol	Union{...}	Any	Union{...}	Any	Int64	DataType
1	year	1982.99	1951	1986.0	2014	0	Int64
2	language		ALGOL 58		dBase III	0	String31

Рисунок 15: Пример создания структуры DataFrame

14 RDatasets

```
# Подгружаем пакет RDatasets:
using RDatasets

✓ 0.2s Julia

# Задаем структуру данных в виде набора данных:
iris = dataset("datasets", "iris")

✓ 6.5s Julia
```

150×5 DataFrame 125 rows omitted

Row	SepalLength	SepalWidth	PetalLength	PetalWidth	Species
	Float64	Float64	Float64	Float64	Cat...
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa
7	4.6	3.4	1.4	0.3	setosa
8	5.0	3.4	1.5	0.2	setosa
9	4.4	2.9	1.4	0.2	setosa
10	4.9	3.1	1.5	0.1	setosa
11	5.4	3.7	1.5	0.2	setosa
12	4.8	3.4	1.6	0.2	setosa
13	4.8	3.0	1.4	0.1	setosa
139	6.0	3.0	4.8	1.8	virginica
140	6.9	3.1	5.4	2.1	virginica
141	6.7	3.1	5.6	2.4	virginica
142	6.9	3.1	5.1	2.3	virginica
143	5.8	2.7	5.1	1.9	virginica
144	6.8	3.2	5.9	2.3	virginica

Рисунок 16: Работа с пакетом RDatasets

15 RDatasets

```
# Определения типа переменной:
```

```
typeof(iris)
```

✓ 0.2s

Julia

DataFrame

```
describe(iris)
```

✓ 0.5s

Julia

5×7 DataFrame

Row	variable	mean	min	median	max	nmissing	eltype
	Symbol	Union...	Any	Union...	Any	Int64	DataType
1	SepalLength	5.84333	4.3	5.8	7.9	0	Float64
2	SepalWidth	3.05733	2.0	3.0	4.4	0	Float64
3	PetalLength	3.758	1.0	4.35	6.9	0	Float64
4	PetalWidth	1.19933	0.1	1.3	2.5	0	Float64
5	Species		setosa		virginica	0	CategoricalValue{String, UInt8}

16 Работа с переменными отсутствующего типа (Missing Values)

```
# Отсутствующий тип:
```

```
a = missing
```

```
typeof(a)
```

✓ 0.0s

Julia

Missing

```
# Пример операции с переменной отсутствующего типа:
```

```
a + 1
```

✓ 0.1s

Julia

missing

17 Работа с переменными отсутствующего типа (Missing Values)

```
# Определение перечня продуктов:  
foods = ["apple", "cucumber", "tomato", "banana"]  
  
# Определение калорий:  
calories = [missing, 47, 22, 105]
```

✓ 0.3s

Julia

```
4-element Vector{Union{Missing, Int64}}:  
 missing  
 47  
 22  
 105
```

```
# Определение типа переменной:  
typeof(calories)
```

✓ 0.0s

Julia

```
Vector{Union{Missing, Int64}} (alias for Array{Union{Missing, Int64}, 1})
```

18 Работа с переменными отсутствующего типа (Missing Values)

```
# Подключаем пакет Statistics:
```

```
using Statistics
```

```
✓ 0.0s
```

Julia

```
# Определение среднего значения:
```

```
mean(calories)
```

```
✓ 0.1s
```

Julia

```
missing
```

```
# Определение среднего значения без значений с отсутствующим типом:
```

```
mean(skipmissing(calories))
```

```
✓ 0.1s
```

Julia

```
58.0
```

19 Работа с переменными отсутствующего типа (Missing Values)

```
# Задание сведений о ценах:
prices = [0.85, 1.6, 0.8, 0.6]

# Формирование данных о калориях:
dataframe_calories = DataFrame(item=foods, calories=calories)

# Формирование данных о ценах:
dataframe_prices = DataFrame(item=foods, price=prices)

# Объединение данных о калориях и ценах:
DF = leftjoin(dataframe_calories, dataframe_prices, on=:item)
```

✓ 2.9s

Julia

4x3 DataFrame

Row	item	calories	price
	String	Int64?	Float64?
1	apple	missing	0.85
2	cucumber	47	1.6
3	tomato	22	0.8
4	banana	105	0.6

20 FileIO

```
# Подключаем пакет FileIO:  
using FileIO  
✓ 0.0s
```

Julia

```
# Подключаем пакет ImageIO:  
import Pkg  
Pkg.add("ImageIO")
```

Julia

```
# Загрузим изображение (в данном случае логотип Julia):  
# Загрузка изображения:  
X1 = load("julia.png")  
✓ 15s
```

Julia

```
Warning: Output swatches are reduced due to the large size (180x280).  
Load the ImageShow package for large images.  
@ Colors C:\Users\durda\.julia\packages\Colors\VF3J1\src\display.jl:159
```



21 Обработка данных: стандартные алгоритмы машинного обучения в Julia.

Кластеризация данных. Метод k-средних

```
# Загрузка данных:
houses = CSV.File("houses.csv") |> DataFrame
✓ 5.0s
```

985 × 12 DataFrame 960 rows omitted

Row	street	city	zip	state	beds	baths	sq_ft	type	sale_date	price	latitude	longitude
	String	String15	Int64	String3	Int64	Int64	Int64	String15	String91	Int64	Float64	Float64
1	3526 HIGH ST	SACRAMENTO	95838	CA	2	1	836	Residential	Wed May 21 00:00:00 EDT 2008	59222	38.6319	-121.435
2	51 OMAHA CT	SACRAMENTO	95823	CA	3	1	1167	Residential	Wed May 21 00:00:00 EDT 2008	68212	38.4789	-121.431
3	2796 BRANCH ST	SACRAMENTO	95815	CA	2	1	796	Residential	Wed May 21 00:00:00 EDT 2008	68880	38.6183	-121.444
4	2805 JANETTE WAY	SACRAMENTO	95815	CA	2	1	852	Residential	Wed May 21 00:00:00 EDT 2008	69307	38.6168	-121.439
5	6001 MCMAHON DR	SACRAMENTO	95824	CA	2	1	797	Residential	Wed May 21 00:00:00 EDT 2008	81900	38.5195	-121.436
6	5828 PEPPERMILL CT	SACRAMENTO	95841	CA	3	1	1122	Condo	Wed May 21 00:00:00 EDT 2008	89921	38.6626	-121.328
7	6048 OGDEN NASH WAY	SACRAMENTO	95842	CA	3	2	1104	Residential	Wed May 21 00:00:00 EDT 2008	90895	38.6817	-121.352
8	2561 19TH AVE	SACRAMENTO	95820	CA	3	1	1177	Residential	Wed May 21 00:00:00 EDT 2008	91002	38.5351	-121.481
9	11150 TRINITY RIVER DR Unit 114	RANCHO CORDOVA	95670	CA	2	2	941	Condo	Wed May 21 00:00:00 EDT 2008	94905	38.6212	-121.271
10	7325 10TH ST	RIO LINDA	95673	CA	3	2	1146	Residential	Wed May 21 00:00:00 EDT 2008	98937	38.7009	-121.443
11	645 MORRISON AVE	SACRAMENTO	95838	CA	3	2	909	Residential	Wed May 21 00:00:00 EDT 2008	100309	38.6377	-121.452
12	4085 FAWN CIR	SACRAMENTO	95823	CA	3	2	1289	Residential	Wed May 21 00:00:00 EDT 2008	106250	38.4707	-121.459
13	2930 LA ROSA RD	SACRAMENTO	95815	CA	1	1	871	Residential	Wed May 21 00:00:00 EDT 2008	106852	38.6187	-121.436
974	2181 WINTERHAVEN CIR	CAMERON PARK	95682	CA	3	2	0	Residential	Thu May 15 00:00:00 EDT 2008	224500	38.6976	-120.996
975	7540 HICKORY AVE	ORANGEVALE	95662	CA	3	1	1456	Residential	Thu May 15 00:00:00 EDT 2008	225000	38.7031	-121.235
976	5024 CHAMBERLIN CIR	ELK GROVE	95757	CA	3	2	1450	Residential	Thu May 15 00:00:00 EDT 2008	228000	38.3898	-121.446
977	2400 INVERNESS DR	LINCOLN	95648	CA	3	2	1358	Residential	Thu May 15 00:00:00 EDT 2008	229027	38.8978	-121.325
978	5 BISHOPGATE CT	SACRAMENTO	95823	CA	4	2	1329	Residential	Thu May 15 00:00:00 EDT 2008	229500	38.4679	-121.445
979	5601 REKLEIGH DR	SACRAMENTO	95823	CA	4	2	1715	Residential	Thu May 15 00:00:00 EDT 2008	230000	38.4453	-121.442

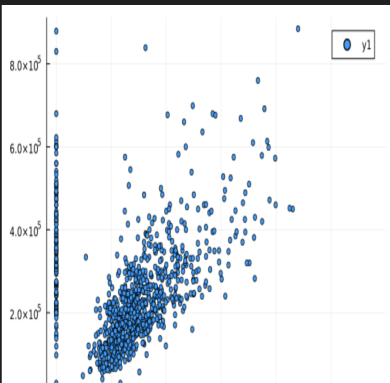
Рисунок 23: Загрузка данных

22 Обработка данных: стандартные алгоритмы машинного обучения в Julia. Кластеризация данных. Метод k-средних

```
# Построение графика:  
plot(size=(500,500),leg=false)  
x = houses[:,sq_ft]  
y = houses[:,price]  
scatter(x,y,markersize=3)
```

✓ 0.6s

Julia

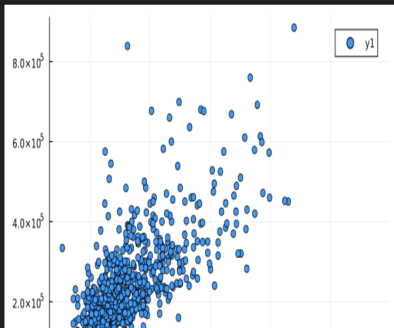


23 Обработка данных: стандартные алгоритмы машинного обучения в Julia. Кластеризация данных. Метод k-средних

```
# Фильтрация данных по заданному условию:  
filter_houses = houses[houses[:,sq_ft].>0,:]  
# Построение графика:  
x = filter_houses[:,sq_ft]  
y = filter_houses[:,price]  
scatter(x,y)
```

✓ 0.2s

Julia



24 Обработка данных: стандартные алгоритмы машинного обучения в Julia. Кластеризация данных. Метод k-средних

```
using Statistics  
  
# Определение средней цены для каждого типа домов  
mean_prices = combine(groupby(filter_houses, :type), :price => mean => :mean_price)  
println("Средние цены для каждого типа домов:")  
println(mean_prices)
```

✓ 12s

Julia

Средние цены для каждого типа домов:

3x2 DataFrame

Row	type	mean_price
	String15	Float64
1	Residential	2.34882e5
2	Condo	1.34213e5

25 Обработка данных: стандартные алгоритмы машинного обучения в Julia. Кластеризация данных. Метод k-средних

```
# Добавление данных : latitude и : Longitude
X = filter_houses[:, [:latitude, :longitude]]

# Преобразование DataFrame в матрицу
X = Matrix{Float64}(X)

k = length(unique(filter_houses[:, :zip]))

# Определение кластеров k-среднего
C = kmeans(X, k)

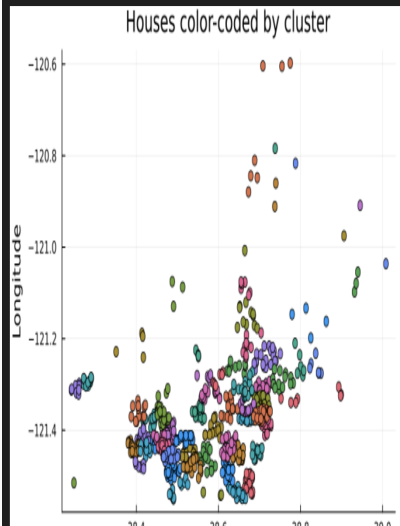
# Формирование DataFrame с результатами кластеризации
df = DataFrame(
    cluster = C.assignments,
    city = filter_houses[:, :city],
    latitude = filter_houses[:, :latitude],
    longitude = filter_houses[:, :longitude],
    zip = filter_houses[:, :zip]
)

# Построение графика кластеров
clusters_figure = plot(legend=false)

for i in 1:k
    clustered_houses = df[df[:, :cluster] .== i, :]
    xvals = clustered_houses[:, :latitude]
    yvals = clustered_houses[:, :longitude]
    scatter!(clusters_figure, xvals, yvals, markersize=4)
end

xlabel!("Latitude")
ylabel!("Longitude")
```

26 Обработка данных: стандартные алгоритмы машинного обучения в Julia. Кластеризация данных. Метод k-средних



27 Обработка данных: стандартные алгоритмы машинного обучения в Julia. Кластеризация данных. Метод k-средних

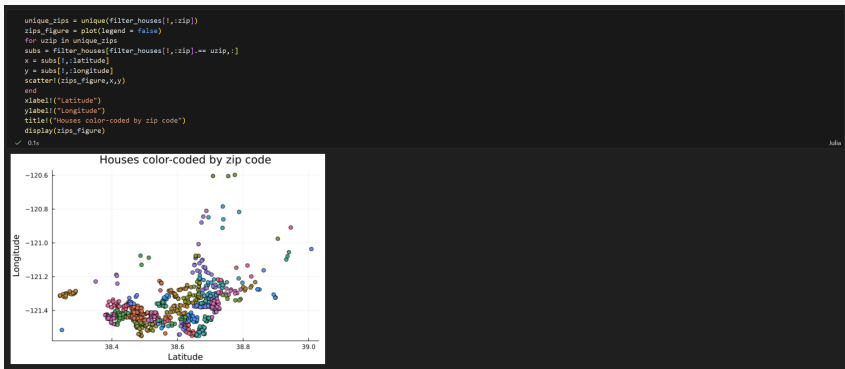


Рисунок 29: Построение графика с кластерами разных цветов по почтовому индексу

28 Кластеризация данных. Метод k ближайших соседей

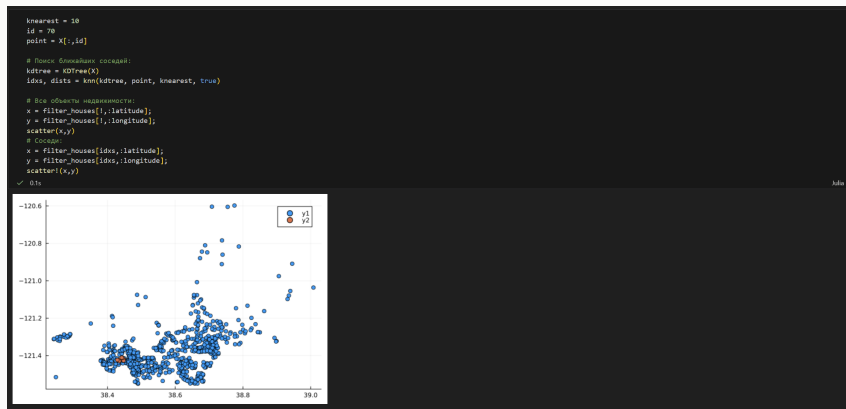


Рисунок 30: Отображение на графике соседей выбранного объекта недвижимости

29 Кластеризация данных. Метод k ближайших соседей

```
# Фильтрация по районам соседних домов:
```

```
cities = filter_houses[idxs,:city]
```

```
✓ 0.5s
```

Julia

```
10-element PooledArrays.PooledVector{String15, UInt32, Vector{UInt32}}:
```

```
"SACRAMENTO"
```

```
"ELK GROVE"
```

```
"SACRAMENTO"
```

```
"SACRAMENTO"
```

```
"SACRAMENTO"
```

```
"SACRAMENTO"
```

```
"ELK GROVE"
```

```
"ELK GROVE"
```

```
"ELK GROVE"
```

```
"ELK GROVE"
```

30 Обработка данных. Метод главных компонент

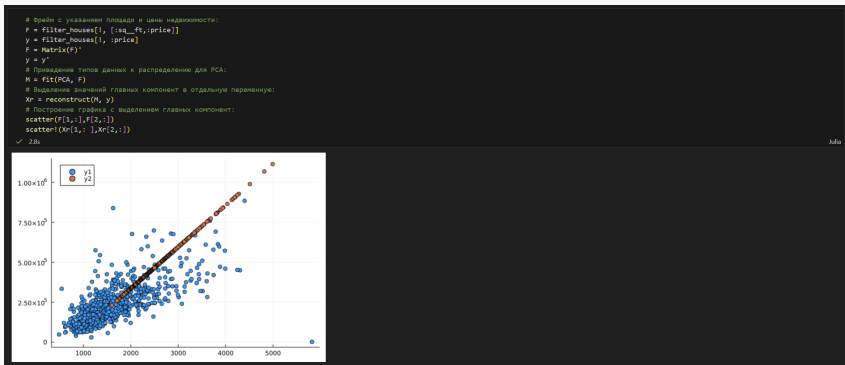


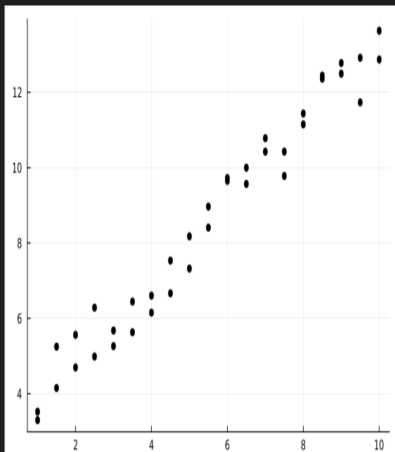
Рисунок 32: Попытка уменьшения размера данных о цене и площади из набора данных домов

31 Обработка данных. Линейная регрессия

```
xvals = repeat(1:0.5:10,inner=2)
yvals = 3 .+ xvals + 2*rand(length(xvals)) .- 1
scatter(xvals,yvals,color=:black,leg=false)
```

✓ 0.2s

Julia



32 Обработка данных. Линейная регрессия

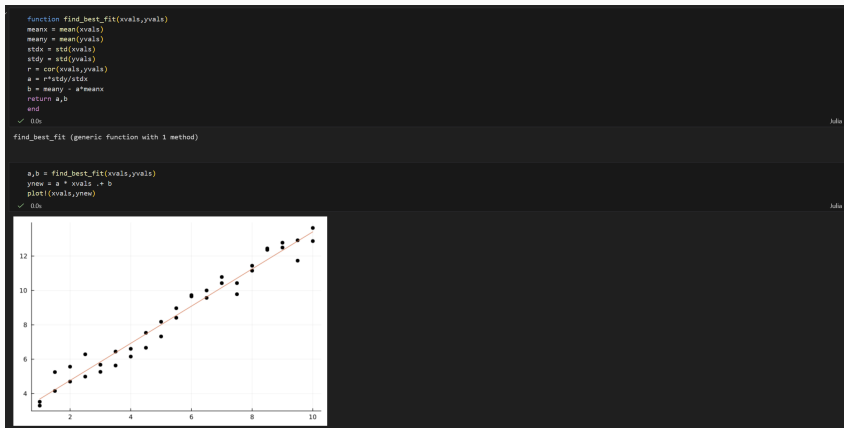


Рисунок 34: Применение функции для построения графика

33 Обработка данных. Линейная регрессия

```
xvals = 1:100000;  
xvals = repeat(xvals,inner=3);  
yvals = 3 .* xvals + 2*rand(length(xvals)) .* 1;  
@show size(xvals)  
@show size(yvals)
```

✓ 0.8s

Julia

```
size(xvals) = (300000,)  
size(yvals) = (300000,)
```

```
(300000,)
```

```
@time a,b = find_best_fit(xvals,yvals)
```

✓ 0.3s

Julia

```
0.023522 seconds (33.26 k allocations: 1.660 MiB, 95.95% compilation time)
```

```
(1.0000000095882504, 2.9993275259039365)
```

```
using PyCall  
using Conda
```

✓ 0.0s

Julia

34 Обработка данных. Линейная регрессия

```
py"""
import numpy
def find_best_fit_python(xvals,yvals):
    meanx = numpy.mean(xvals)
    meany = numpy.mean(yvals)
    stdx = numpy.std(xvals)
    stdy = numpy.std(yvals)
    r = numpy.corrcoef(xvals,yvals)[0][1]
    a = r*stdy/stdx
    b = meany - a*meanx
    return a,b
"""

xpy = PyObject(xvals)
ypy = PyObject(yvals)
@time a,b = py"find_best_fit_python"(xpy,ypy)
```

✓ 0.3s

Julia

0.138421 seconds (138.93 k allocations: 6.838 MiB, 45.05% compilation time)

(1.0000000095882489, 2.999327525983972)



```
using BenchmarkTools
@btime a,b = py"find_best_fit_python"(xvals,yvals)
@btime a,b = find_best_fit(xvals,yvals)
```

✓ 25.6s

Julia

4.182 ms (34 allocations: 960 bytes)

398.700 μs (1 allocation: 32 bytes)

(1.0000000095882489, 2.999327525983972)

35 Самостоятельное выполнение

Задания

1.1) Кластеризация

🔗 Ссылка + Code + Markdown

using RDatasets, Clustering, Plots

✓ 00s

Julia

```
iris = dataset("datasets", "iris")
iris_spec = Dict{"setosa" => 1, "versicolor" => 2, "virginica" => 3}
iris[1, :5] = [iris_spec[item] for item in iris[1, :5]]

X = Matrix{iris}
k = length(unique(iris[1, :5]))
C = kmeans(X, k)

df = DataFrame(cluster = C.assignments, Sepallength = iris[1, :Sepallength],
               Sepalwidth = iris[1, :Sepalwidth], Petallength = iris[1, :Petallength],
               Petalwidth = iris[1, :Petalwidth], Species = iris[1, :Species])

clusters_figure = plot(legend = false)
for i = 1:k
    clustered_houses = df[df[1, :cluster].== i,:]
    xvals = clustered_houses[1, :Sepalwidth]
    yvals = clustered_houses[1, :Sepallength]
    scatter!(clusters_figure, xvals, yvals, markersize = 4)
end
xlabel!("Sepalwidth")
ylabel!("Sepallength")
title!("Iris. Cluster")
display(clusters_figure)
```

✓ 76s

Julia

Рисунок 37: Кластеризация

36 Самостоятельное выполнение

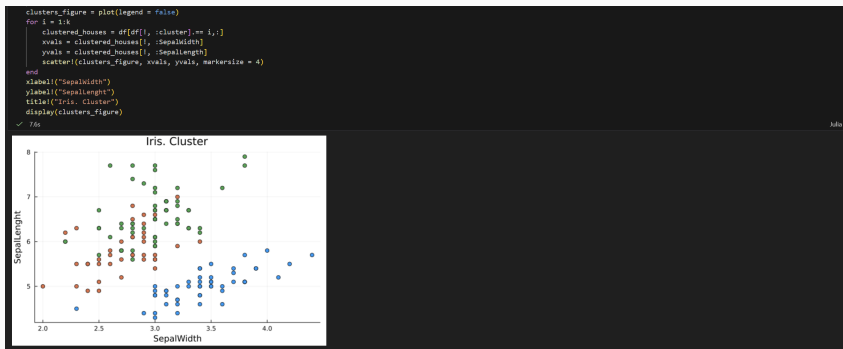


Рисунок 38: Кластеризация

37 Самостоятельное выполнение

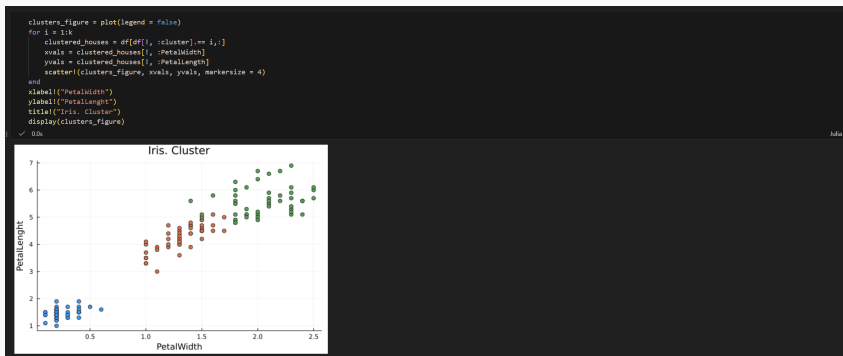


Рисунок 39: Кластеризация

38 Самостоятельное выполнение

1.2) Регрессия (МНК в случае линейной регрессии)

Часть 1.

[% Создать](#) [+ Code](#) [+ Markdown](#)

```
X = randn(1000, 3)
a0 = rand(3)
y = X * a0 + 0.1 * randn(1000)
scatter(X, y, vals = (1, 2, 3))
```

✓ 0.2s

Julia

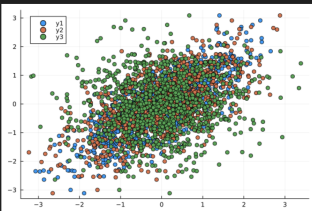


Рисунок 40: Регрессия

39 Самостоятельное выполнение

```
X2 = hcat(ones(1000), X)
beta = (X2' * X2) \ (X2' * y)
✓ 0.0s Julia

4-element Vector{Float64}:
-0.002839306466767834
0.7906166914006252
0.5988088828846224
0.0963053041862317

using MultivariateStats
beta_mvs = llsq(X, y)
✓ 0.2s Julia

4-element Vector{Float64}:
0.7906166914006254
0.5988088828846223
0.09630530418623166
-0.0028393064667678277

using GLM
df = DataFrame(hcat(X, y), :auto)
rename!(df, ["x1", "x2", "x3", "y"])
model = lm(@formula(y ~ x1 + x2 + x3), df)
coef(model)
✓ 0.0s Julia

4-element Vector{Float64}:
-0.0028393064667678273
0.7906166914006253
0.5988088828846224
0.09630530418623173
```

Рисунок 41: Регрессия

40 Самостоятельное выполнение

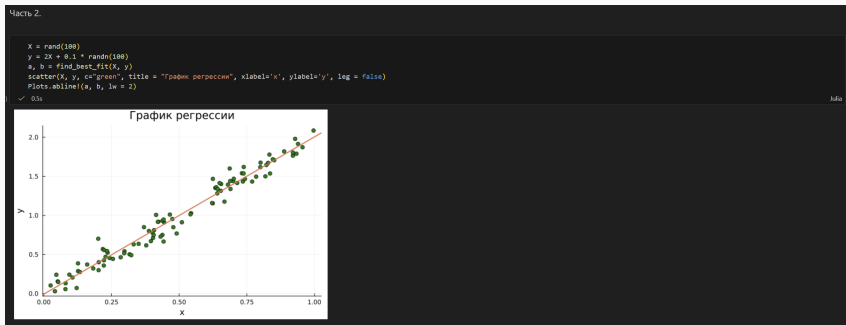


Рисунок 42: Регрессия

41 Самостоятельное выполнение

1.3) Модель ценообразования биномиальных опционов

а.

```
S = 100.0 # начальная цена акции
T = 1.0   # длина биномиального дерева в годах;
n = 10000 # количество периодов
sigma = 0.3 # волатильность
r = 0.08   # годовая процентная ставка
h = T / n  # длина одного периода

u = exp(r * h + sigma * sqrt(h))
d = exp(r * h - sigma * sqrt(h))
p = (exp(r*h) - d) / (u - d)

function create_path(S, r, sigma, T, n)
    path = []
    S_ = S
    push!(path, S)
    for i in 1:n
        if rand() < p
            S_ *= u
        else
            S_ *= d
        end
        push!(path, S_)
    end
    return path
end

path = create_path(S, r, sigma, T, n)
plot(path, label="Stock Price Path", xlabel="Time Step", ylabel="Price", title="Stock Price Trajectory")
```

✓ 02s

julia

Рисунок 43: Модель ценообразования биномиальных опционов

42 Самостоятельное выполнение

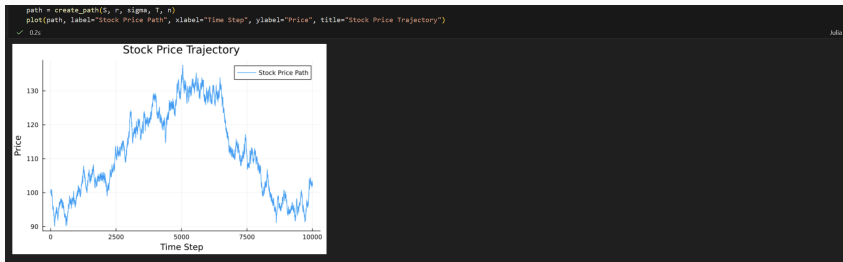


Рисунок 44: Модель ценообразования биномиальных опционов

43 Самостоятельное выполнение

b.

```
function createPath(S::Float64, r::Float64, sigma::Float64, T::Float64, n::Int64)
    Y = Vector{Float64}()
    X = collect{Float64}(0:T/n:1)
    u = exp(r * h + sigma * sqrt(h))
    d = exp(r * h - sigma * sqrt(h))
    p = (exp(r*h) - d) / (u - d)
    for i=0:n
        prob = rand()
        if prob < p
            S *= u
        else
            S *= d
        end
        push!(Y, S)
    end
    return(X, Y)
end

p = plot()
for i = 1:10
    plot!(p, createPath(S, r, sigma, T, n))
end
end
p
```

✓ 0.6s julia

Рисунок 45: Модель ценообразования биномиальных опционов

44 Самостоятельное выполнение

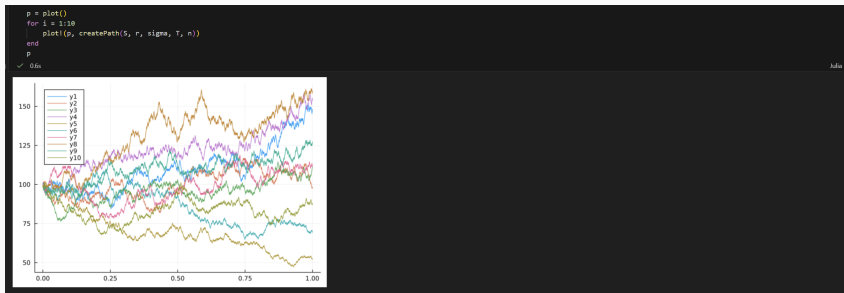


Рисунок 46: Модель ценообразования биномиальных опционов

45 Самостоятельное выполнение

```
C.  
  
function createPath(S::Float64, r::Float64, sigma::Float64, T::Float64, n::Int64)  
    Y = Vector{Float64}()  
    X = collect{Float64}(0:n-1)  
    u = exp(r * h + sigma * sqrt(h))  
    d = exp(r * h - sigma * sqrt(h))  
    p = (exp(r*h) - d) / (u - d)  
    Threads.@threads for i=0:n  
        prob = rand()  
        if prob < p  
            S *= u  
        else  
            S *= d  
        end  
        push!(Y, S)  
    end  
    return(X, Y)  
end  
  
p = plot()  
for i = 1:10  
    plot!(p, createPath(S, r, sigma, T, n))  
end  
p  
p
```

0.3s Julia

Рисунок 47: Модель ценообразования биномиальных опционов

46 Самостоятельное выполнение

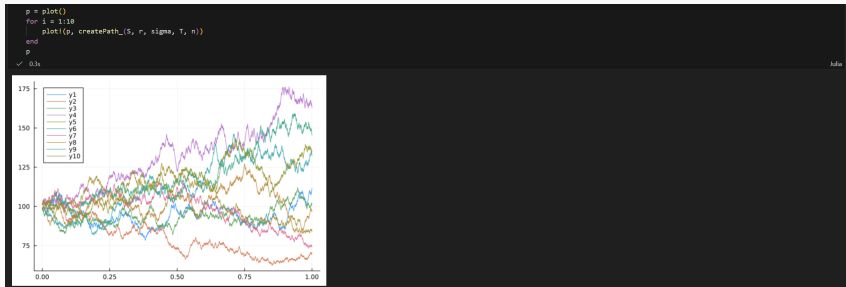


Рисунок 48: Модель ценообразования биномиальных опционов

В ходе выполнения лабораторной работы я изучил специализированные пакеты Julia для обработки данных.